

Липецкий государственный технический университет

Институт компьютерных наук  
Кафедра прикладной математики и системного анализа

Отчет по лабораторной работе № 2  
по дисциплине «Нейросетевые методы искусственного интеллекта»  
на тему «Прямое и обратное распространение ошибки в нейронных сетях»

|                               |               |                   |
|-------------------------------|---------------|-------------------|
| Студент                       |               | Сараев А.П.       |
|                               | подпись, дата | фамилия, инициалы |
| Группа ПМ-24-1                |               |                   |
| Руководитель                  |               | Попов Р.В.        |
| К.Т.Н.                        |               |                   |
| учёная степень, учёное звание | подпись, дата | фамилия, инициалы |

# 1 Цель работы

Изучить и практически реализовать алгоритмы прямого и обратного распространения ошибки для обучения многослойного перцептрона. Исследовать влияние архитектуры сети (число скрытых слоёв и нейронов) и скорости обучения на процесс минимизации функции потерь. Продемонстрировать необходимость нелинейных скрытых слоёв для решения задач, которые не являются линейно разделимыми.

## 2 Задание

Необходимо реализовать обучение нейронной сети с использованием только алгоритмов прямого и обратного распространения ошибки (без использования готовых слоёв и оптимизаторов из библиотек глубокого обучения, таких как `torch.nn` или `tensorflow.keras`). Разрешается использовать библиотеки для работы с массивами (`numpy`) и для генерации данных (`sklearn.datasets`).

## 3 Выбор датасета и подготовка данных

В качестве датасета выбран **Two Moons** (`make_moons`). Классы линейно неразделимы.

Листинг 1: Исходный датасет

```
X, Y = make_moons(n_samples = 600, noise = 0.2, random_state=42)
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size = 0.3)
```

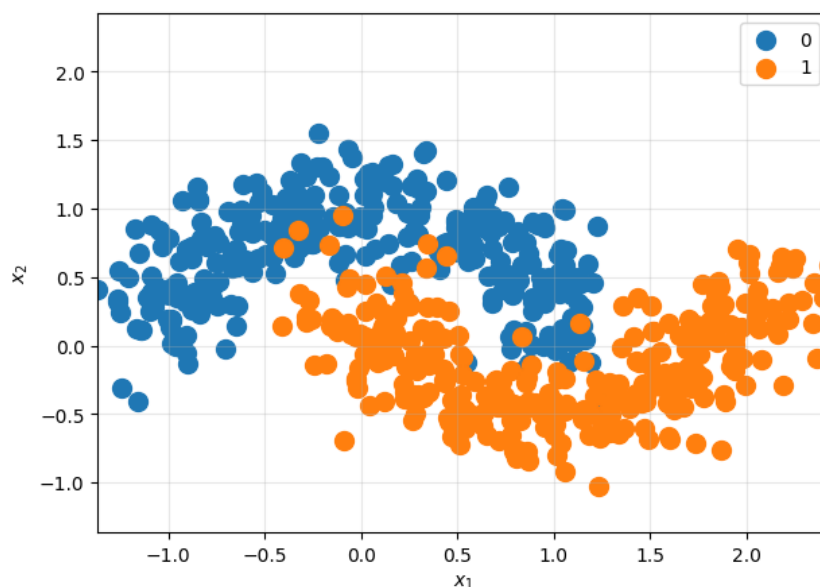


Рис. 1: Визуализация исходных данных

## 4 Реализация нейронной сети и исследование архитектур

### 4.1 Реализация нейронной сети

Листинг 2: Математические функции

```
def tanh(x):
    return (math.exp(x) - math.exp(-x)) / (math.exp(x) + math.exp(-x))

def tanh_dif(x):
    return 1 - tanh(x) ** 2

def sigmoid(x):
    return 1 / (1 + math.exp(-x))

def sigmoid_dif(x):
    return sigmoid(x) * (1 - sigmoid(x))

def MSE(y_calc, y_real):
    total = 0
    for i in range(len(y_real)):
        total += (y_calc[i][0] - y_real[i]) ** 2
    return total / 2

def MSE_dif(y_calc, y_real):
    # sum = 0
    # for i in range(0, len(y_real)):
    #     sum += (y_calc[i] - y_real[i])
    # return [sum]
    return [y_calc[0] - y_real]
```

Листинг 3: Класс нейрона

```
class Neuron:
    def __init__(self, num, activation = "sigmoid"):
        self.activation = activation
        self.weights = np.random.randn(num)
        self.bias = 0
        self.input = []
        self.output = 0
        self.dL_dw = []
```

```

self.dL_db = 0

def forward(self, input):
    self.input = input

    self.output = 0
    for i in range(len(self.input)):
        self.output += self.weights[i] * self.input[i]
    self.output += self.bias
    self.z = self.output
    self.output = (sigmoid(self.output) if self.activation == 'sigmoid' else tanh(
self.output))

    return self.output

def backward(self, dL_da):
    mul = dL_da * (sigmoid_dif(self.z) if self.activation == 'sigmoid' else
tanh_dif(self.z))
    dL_dx = [mul * i for i in self.weights]
    self.dL_dw = [mul * i for i in self.input]
    self.dL_db = mul
    return dL_dx

def update(self, learning_rate):
    self.weights = [self.weights[i] - learning_rate * self.dL_dw[i] for i in range
(len(self.weights))]
    self.bias = self.bias - learning_rate * self.dL_db

```

Листинг 4: Класс слоя

```

class Layer:
    def __init__(self, input_size, output_size, activation = "sigmoid"):
        self.neurons = [Neuron(input_size, activation) for i in range(output_size)]
        self.input_size = input_size
        self.output_size = output_size
        self.outputs = []

    def forward(self, input):
        self.outputs = []
        for neuron in self.neurons:
            self.outputs.append(neuron.forward(input))
        return self.outputs

```

```

def backward(self, dL_da):
    dL_dx = [0] * self.input_size
    for i in range(self.output_size):
        dL_dx_cur = self.neurons[i].backward(dL_da[i])
        dL_dx = [dL_dx[j] + dL_dx_cur[j] for j in range(self.input_size)]
    return dL_dx

def update(self, learning_rate):
    for neuron in self.neurons:
        neuron.update(learning_rate)

```

#### Листинг 5: Класс сети

```

class Network:
    def __init__(self):
        self.layers = []
        self.loss = []

    def addLayer(self, layer):
        self.layers.append(layer)

    def forward(self, input):
        cur_input = input
        for layer in self.layers:
            cur_input = layer.forward(cur_input)
        return cur_input

    def backward(self, output):
        dL_da = MSE_dif(self.layers[-1].outputs, output)
        for i in range(len(self.layers) - 1, -1, -1):
            dL_da = self.layers[i].backward(dL_da)

    def predict(self, input):
        return self.forward(input)

    def update(self, learning_rate):
        for layer in self.layers:
            layer.update(learning_rate)

    def train(self, x_train, y_train, epoch = 100, learning_rate = 0.001):
        print('=====')

```

```

for i in range(epoch):
    for x, y in zip(x_train, y_train):
        self.forward(x)
        self.backward(y)
        self.update(learning_rate)

    prediction = [self.forward(x_train[i]) for i in range(len(x_train))]
    loss = MSE([prediction[i] for i in range(len(prediction))], [y_train[i] for
i in range(len(y_train))])
    self.loss.append(loss)
    if i % (epoch // 10) == 0:
        print(f"Epoch {i + 1}:")
        print(f"Loss: {loss}")
    print(f"Epoch {epoch}:")
    print(f"Loss: {loss}")
    print('=====')

def get_loss(self):
    return self.loss

```

## 4.2 Исследование архитектур

Листинг 6: Инициализация архитектур

```

models_layers = [[2, 1],
                  [2, 2, 1],
                  [2, 8, 1],
                  [2, 8, 8, 1]]

models = []
for m in models_layers:
    np.random.seed(42)
    network = Network()
    for i in range(1, len(m)):
        if i + 1 == len(m):
            network.addLayer(Layer(m[i - 1], m[i], 'sigmoid'))
        else:
            network.addLayer(Layer(m[i - 1], m[i], 'tanh'))
    models.append(network)

```

Листинг 7: Обучение моделей

```

epoch = 5000
learning_rate = 0.01

fig, ax = plt.subplots(nrows=2, ncols=2, figsize=(8,8))
axes = ax.flatten()

for i in range(4):
    print(f"Model with layers {models_layers[i]}")
    models[i].train(X_train, Y_train, epoch = epoch, learning_rate = learning_rate)
    train_loss = models[i].get_loss()

```

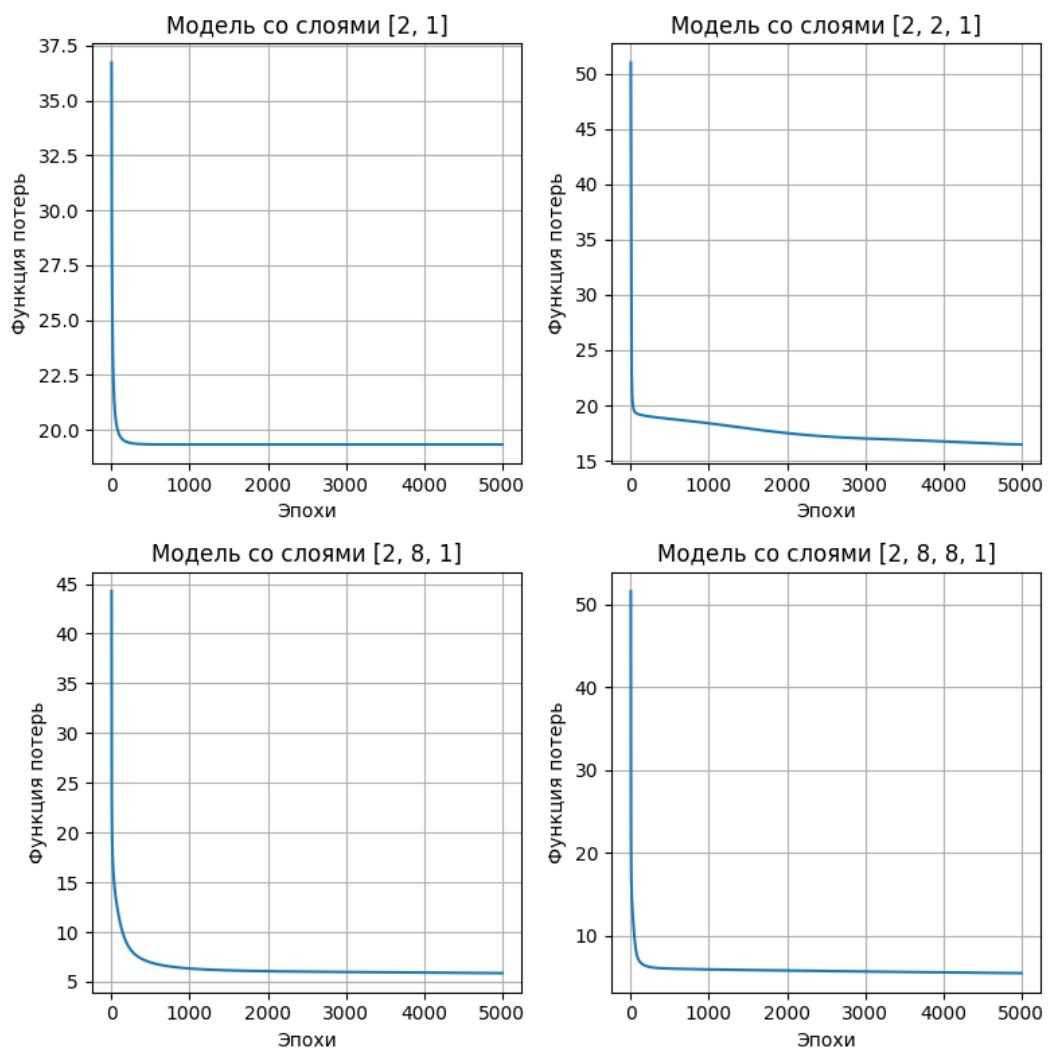


Рис. 2: Графики функции потерь от эпох

| Модель       | Train | Test  |
|--------------|-------|-------|
| [2, 1]       | 0.879 | 0.867 |
| [2, 2, 1]    | 0.869 | 0.883 |
| [2, 8, 1]    | 0.969 | 0.972 |
| [2, 8, 8, 1] | 0.971 | 0.961 |

Таблица 1: Точность моделей

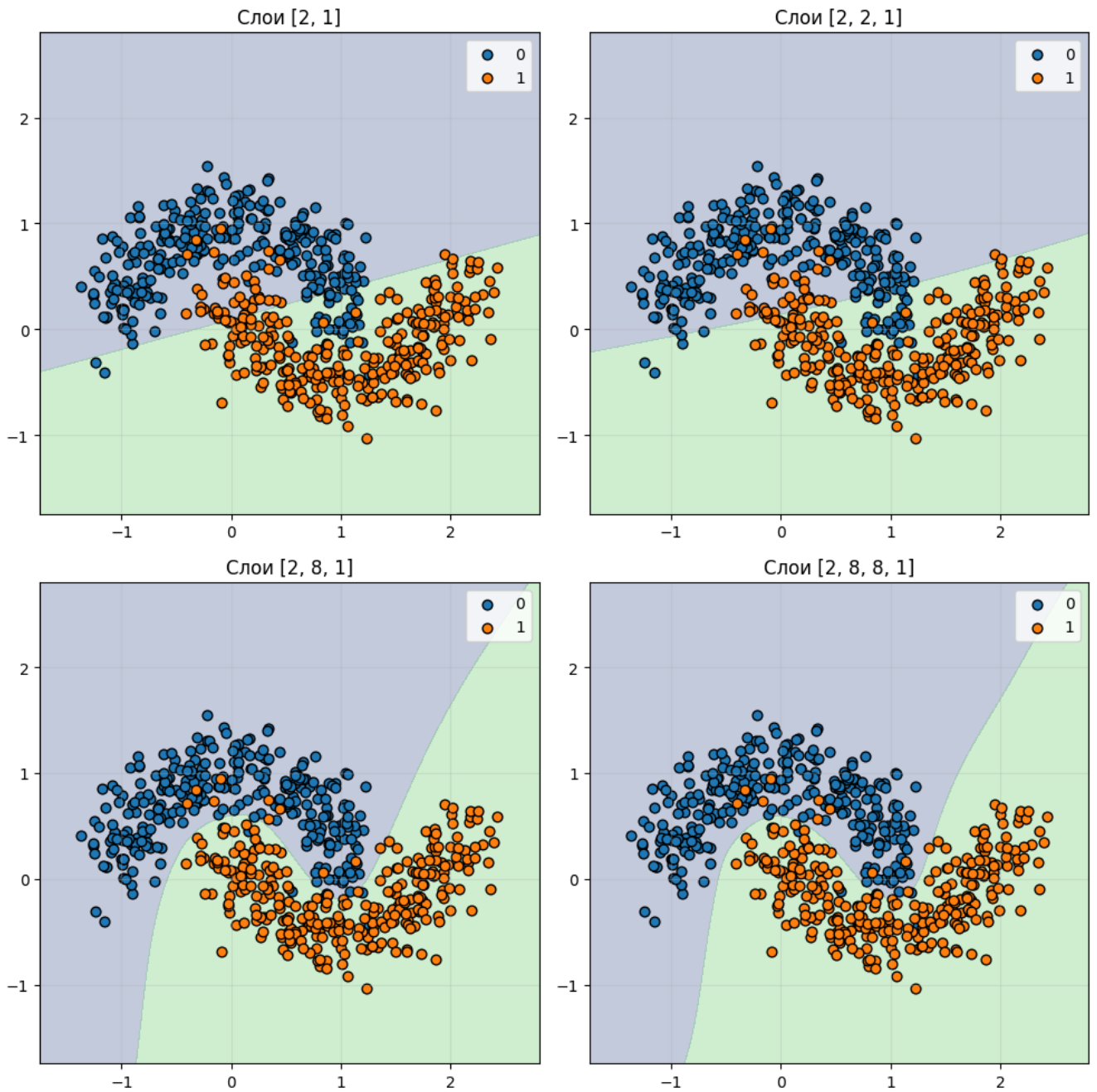


Рис. 3: Визуализация решающих границ

## 5 Исследование скорости обучения (learning rate)

Лучшая модель со слоями [2, 8, 1]. Её точность сравнима с точностью модели [2, 8, 8, 1], но эта (вторая) модель имеет большее число слоёв, нейронов и связей, она будет медленнее.

Сравним обучение модели [2, 8, 1] с learning rate:  $r \in \{0.001, 0.01, 0.1, 1.0\}$ .

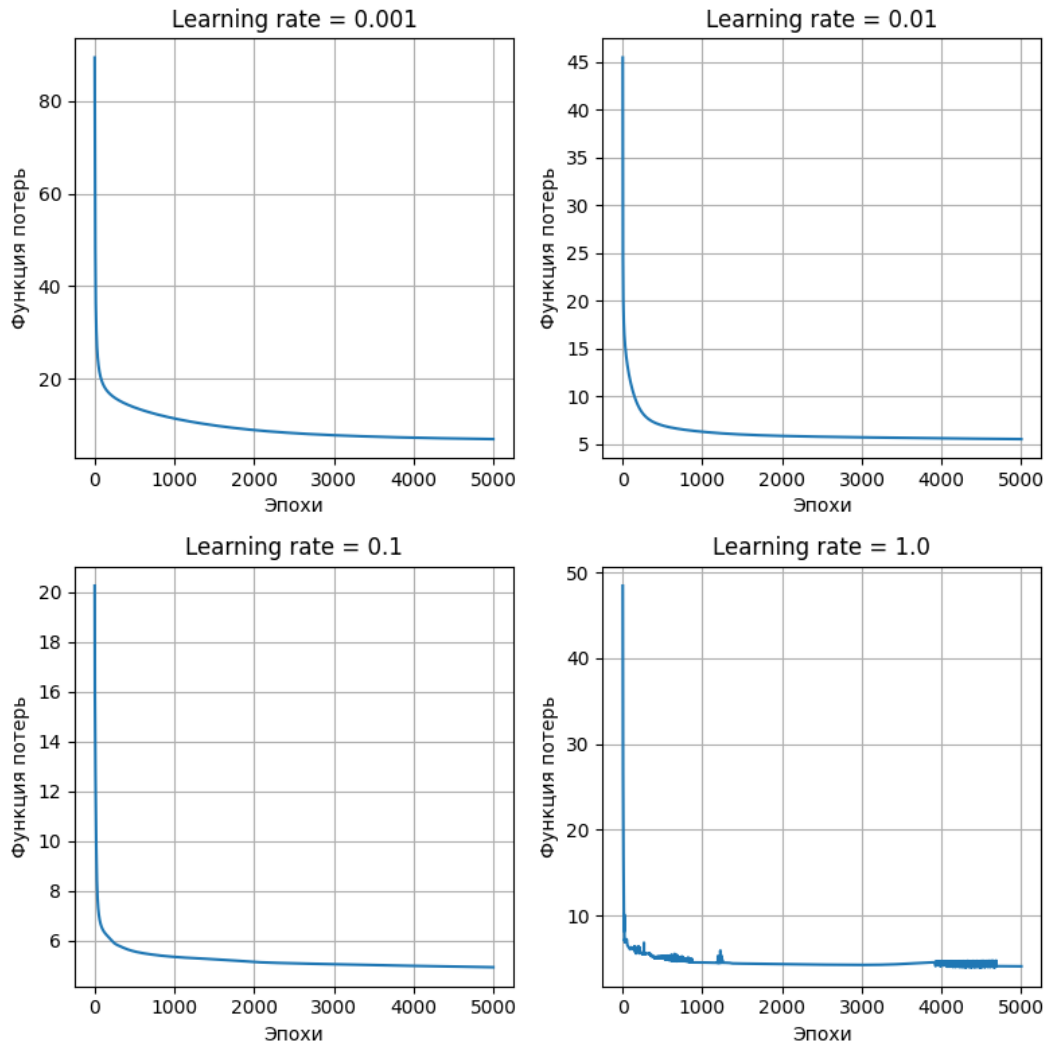


Рис. 4: Графики функций потерь для моделей с разными learning rate

| $r$   | Эпоха с<br>Loss < 5 | Loss на<br>эпохе 500 | Loss на<br>эпохе 1000 | Loss на<br>эпохе 2000 | Loss на<br>эпохе 5000 |
|-------|---------------------|----------------------|-----------------------|-----------------------|-----------------------|
| 0.001 | —                   | ~13.8                | ~11.4                 | ~8.9                  | ~7.07                 |
| 0.01  | —                   | ~6.97                | ~6.3                  | ~5.88                 | ~5.54                 |
| 0.1   | ~3500               | ~5.56                | ~5.3                  | ~5.1                  | ~4.93                 |
| 1.0   | ~500                | ~5.05                | ~4.5                  | ~4.3                  | ~4.39                 |

Таблица 2: Сравнение скоростей обучения

При увеличении learning rate скорость выхода на плато возрастает:  $\eta=0.1$  и  $\eta=1.0$  достигают плато за первые 500 эпох,  $\eta=0.01$  — примерно к 1500-й, а  $\eta=0.001$  подходит к нему лишь к концу 5000 эпох. Однако, при learning rate=1.0 можно заметить, что функция потерь теряет монотонность, ее значения (а значит и график) начинают с определённого момента иногда колебаться, из-за чего график получается не гладким, а со скачками.

## 6 Выводы и анализ результатов

Линейная модель (без скрытых слоёв) не справляется с линейно неразделимыми данными:

1. Из графика с решающей границей мы видим, что набор данных не линейный и его невозможно разделить на две группы одной прямой линией;
2. В сравнении точности линейной модели с другими видно, что она уступает другим и на 10 п.п. на тестовой выборке.

Увеличение числа нейронов и слоёв повышает выразительную способность сети. С увеличением нейронов слоёв мы видим:

1. На графиках решающих границ, эти границы уже не просто прямые, но кривые, которые лучше разделяют области данных на группы;
2. Точности моделей [2, 8, 1] и [2, 8, 8, 1] лучше моделей [2, 1] и [2, 2, 1].

Почему для данной задачи необходима нелинейная функция активации? При обучении моделей используется алгоритм обратного распространения и идея градиентного спуска. Градиент — это вектор частных производных, следовательно, для его вычисления необходима дифференцируемость. При этом функция должна также имитировать ступенчатую функцию (функция Хевисайда не является дифференцируемой).

Вывод о влиянии скорости обучения на сходимость градиентного спуска. Малые  $\eta$  дают стабильное, но медленное обучение; большие — быстрое, но менее устойчивое.